private ArrayList<Integer> ll = new ArrayList<>();

public void add(int item) {
ll.add(item);
upheapify(ll.size()-1);
}

private void upheapify(int ci) {
// TODO Auto-generated method stub

}

private void downheapify(int pi) {
// TODO Auto-generated method stub
int lci = 2 * pi + 1;
int rci = 2 * pi + 2;
int mini = pi;
if (ll.get(lci) < ll.get(mini)) {
mini=lci;
}
if (ll.get(rci) < ll.get(mini)) {
mini=rci;
}

}

[3,2,3,1,2,4,5,5,6,1,2,3], k = 4

[3,2,3,1,2,4,5,5,6,1,2,3], k = 4
public static void main(String[] args) {
// TODO Auto-generated method stub
int[] arr = { 3, 2, 3, 1, 2, 4, 5, 5, 6 };
int k = 4;
PriorityQueue<Integer> pq = new PriorityQueue<>();
for (int i = 0; i < k; i++) {
pq.add(arr[i]);
}
for (int i = k; i < arr.length; i++) {
if (arr[i] > pq.peek()) {
pq.poll();
pq.add(arr[i]);
}
}
System.out.println(pq.peek());
}

public ListNode mergeKLists(ListNode[] lists) { PriorityQueue<ListNode> pq = new PriorityQueue<>(); for (int i = 0; i < lists.length; i++) {
if (lists[i] != null) {
pq.add(lists[i]);
}
}
ListNode Dummy = new ListNode();
ListNode temp = Dummy;
while (!pq.isEmpty()) {
ListNode rn = pq.poll();
Dummy.next = rn;
Dummy = Dummy.next;
if (rn.next!= null) {
pq.add(rn.next);
}
}

}

Arrays.sort(intervals, (a, b) -> a[0] - b[0]);

public static int minMeetingRooms(int[][] intervals) {
Arrays.sort(intervals, (a, b) ->
a[0] - b[0]);
PriorityQueue<int []> pq = new
PriorityQueue<>((a,b)->a[1]-b[1]);
pq.add(intervals[0]);
for (int i = 1; i
< intervals.length; i++) {
if(intervals[i][0]>pq.peek()[1])
{

}
}

}

public static int minMeetingRooms(int[][] intervals) { Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
PriorityQueue<int []> pq = new PriorityQueue<>((a,b)->a[1]-b[1]); pq.add(intervals[0]);
for (int i = 1; i < intervals.length; i++) { if(intervals[i][0]>pq.peek()[1]) {

}
}

}

Lec-34

21/03/26 12:54 PM